



전자전기컴퓨터설계실험II

실험 전 보고서

LAB2

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 09월 19일
대상	순차논리 실습
검증 환경	Vivado XSim 2026.1 / VS Code VaporView

1. 목적과 관련 이론

실험 목적

순차 논리 회로 8개를 VS Code에서 먼저 검증하고 Vivado와 보드에 확인한다. 클록, 저장, 상태 전이를 파형으로 설명한다.

관련 이론

-동기 리셋과 enable

조건	다음 상승 에지의 동작
rst = 1	초기값으로 변경
rst = 0, enable = 1	회로가 정한 상태 갱신
rst = 0, enable = 0	이전 상태 유지

리셋을 올린 순간과 그 다음 상승 에지를 파형에서 구분한다. 각 모듈의 리셋 우선순위를 읽는다. Mealy의 조합 출력은 입력에도 의존하다.

-nonblocking은 이전 값을 전달한다.

-버튼을 클록으로 쓰기 전에 주의점

기계식 버튼은 한 번 눌러도 접점이 여러 번 될 수 있다. FPGA 클록과도 동기화되어 있지 않다. 최신판의 입력 처리 순서는 동기화 -> 안정 시간 확인 -> 한 클록 펄스다. 동기화 플립플롭 두 단계는 메타안정성이 전달될 위험을 낮춘다. 디바운스는 접점의 반복 변화를 걸러낸다. 시뮬레이션은 메타안정성의 아날로그 현상 자체를 검증하지 않는다. 버튼 잡음 자극과 펄스 수를 검사한다.

-clock-enable과 분주 출력

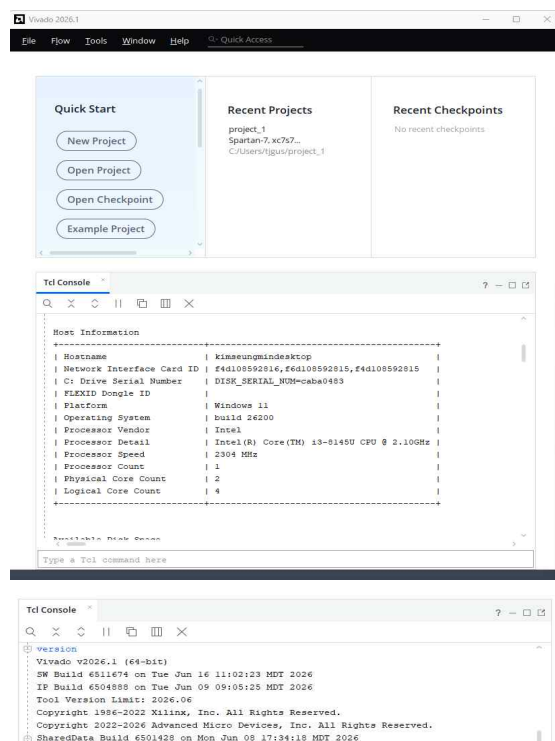
분주 출력은 느린 주기를 LED나 파형으로 관찰할 때 사용한다. 다른 회로의 저장소는 공통 clk를 유지하고, tick이 1인 상승 에지에서만 동작시킨다. DIVISOR = 10이면 tick이 열 클록마다 한 번 소비된다. tick은 소비할 에지 직전부터 1이다. 실제 클록 주파수와 분주 수의 비가 원하는 속도인지 계산한다.

실행 방법

1. 공개 템플릿을 복제하고 새 창에서 연다.
2. RTL, TB, XDC를 직접 작성하고 네 입력 조합을 검사한다.
3. 같은 소스를 Vivado에 등록하고 다시 시뮬레이션한다.
4. Spartan-7용 비트스트림을 생성한다.

실행 환경

- Vivado 2026.1 설치와 실행 확인



- GitHub 계정 생성 및 주소 기재

계정: thechef030

주소: <https://github.com/thechef030>

공통된 도구 버전

icarus verilog: version 13.0

git: version 2.55.0.windows.5

python: python 3.13.15

vscode: 1.137.0

시간을 읽는 기준

TB의 클록 주기는 10ns이다. 상승 에지는

5, 15, 25, ... ns에 온다. 입력을 바꿔도 레지스터는 다음 상승 에지에서 갱신된다. TB는 상승 에지 뒤 1ns 기다린 뒤 결과를 비교한다. nonblocking 갱신 전의 값을 잘못 검사하지 않기 위해서다. 시뮬레이션의 10ns와 실제 장비의 클록 설정은 별개다.

2. 실험 1 업/다운 카운터

회로의 상태표, 리셋, enable 출력 시점

rst	enable	down	다음상태	동작
1	무관	무관	0	리셋
0	0	무관	Q	유지
0	1	0	Q+1	증가
0	1	1	Q-1	감소

리셋은 6ns에 1에서 0으로, 346ns에 0에서 1로 출력이 되고, enable은 6ns에 0에서 1로, 326ns에 1에서 0으로, 다시 336ns에 0에서 1로 출력된다.

주요 RTL과 TB의 역할

- counter4.v

실제 카운터 상태를 저장한다. 상승 에지에서 리셋, 유지, 증가, 감소를 수행한다.

- input_frontend.v

외부 입력을 내부 회로에 전달하기 위한
입력부다. 버튼과 스위치를 두 단계로 동기
화하고 버튼 입력에 디바운스를 적용해 한
클럭의 길이의 press 신호를 만든다.

- lab2 counter.v

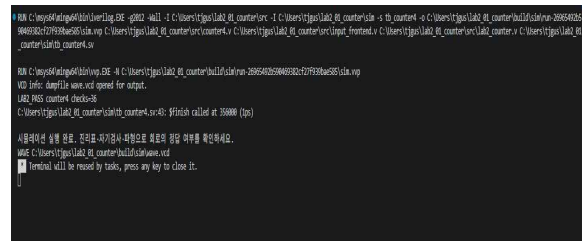
보드용 최상위 모듈이다. 입력부와 카운터를 연결하고 카운터 값을 LED로 출력한다.

- tb counter4.sv

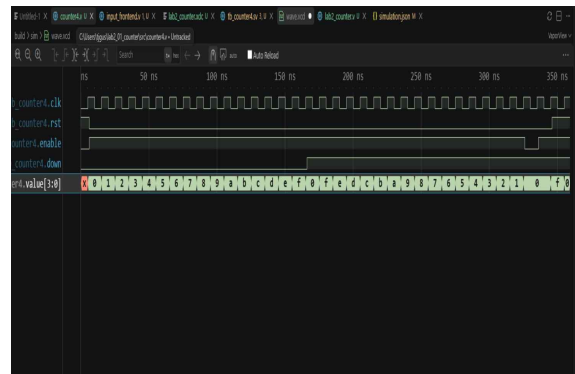
시뮬레이션용 클록과 입력을 만들고 출력
이 예상값과 일치하는지 자동 검사한다.

pass 로그와 전체 파형

-pass 로그



-전체 파형



상승 에지 전후를 확대한 VS Code 파형을 이용한 예상값

테스트벤치는 `timescale 1ns/1ps를 사용하고, clk=0에서 시작해 always #5 clk=~clk로 클럭을 반전시킨다. 따라서 시뮬레이션 클럭 주기는 10ns, 상승 에지는 5ns, 15ns, 25ns 등의 시각에 발생한다. step은 상승 에지 1ns 뒤에 검사하므로, 5ns에서 갱신된 값은 6ns에서 검사한다.

상승에지 시각	입력 조건	값의 변화	동작
5ns	rst=1	x->0	6ns리셋
15ns	rst=0 enable=1 down=1	0->1	16ns 증가
165ns	enable=1 down=0	15->0	166ns 증가순환
175ns	enable=1 down=1	0->15	176ns 감소순환
335ns	rst=0 enable=0	0->0	336ns 유지
345ns	rst=0 enable=1 down=1	0->15	346ns 다시감소

Vivado에 등록할 RTL, TB, XDC, top 장비 클록 조건

- Design Sources: src/counter4.v,
src/input_frontend.v, src/lab2_counter.v
- Simulation Sources: sim/tb_counter4.sv
- Constraints: constraints/lab2_counter.xdc
- 설계 최상위 모듈: lab2_counter
- 시뮬레이션 최상위 모듈: tb_counter4
- 보드 메인 클럭: 1 kHz, 주기 1ms
- TB 클럭: 100 MHz, 주기 10ns

3. 실험 2 클록 분주

count, divided, tick의 예상상태와 분주주기

- divided: 입력 주파수를 DIVISOR로 나눈 사각파이다. 짝수 DIVISOR에서 High와 Low 시간이 같다.
- tick: count가 DIVISOR-1일 때만 1인 한 클럭 폭의 신호이다. 다른 순차회로의 enable로 사용할 수 있다.
- DIVISOR=10일 때 예상 동작

TB에서는 입력 클록 주기를 10ns로 설정한다. 따라서 출력 클록의 주기는 $10 \times 10\text{ns}$ 로 100ns가 된다. 짝수인 10으로 분주하므로 High와 Low의 시간은 각각 50ns이다. DIVISOR=10일 때 예상상태는 다음과 같다.

예지 뒤 count	divided	tick
0-4	0	0
5-8	1	0
9	1	1
다음 예지: 0	0	0

주요 RTL과 TB의 역할

- clock_divider.v
실제 클럭 분주 기능을 수행한다. 내부의 count가 매 입력 클럭 상승 에지마다 증가하며 count가 DIVISOR/2-1에 도달하면 divided를 1로 만들고 DIVISOR-1에 도달하면

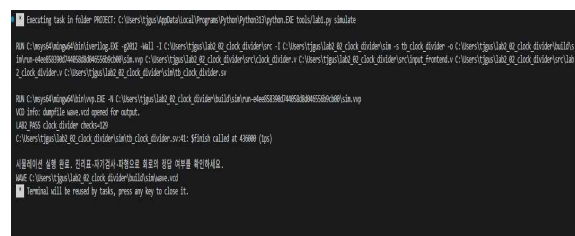
다시 0으로 만든다.

- input_frontend.v
- 보드의 외부 입력을 메인 클록 영역에서 안정적으로 사용하기 위한 입력을 처리한다. stable cycles 동안 입력이 안정된 경우에만 외부 입력을 받아들인다. 안정화 시간은 20ms이다.
- lab2_clock_divider.v
- FPGA 보드에서 사용되는 최상위 모듈이다. 하나의 입력 클록을 이용하여 DIVISOR = 2, 10, 50, 1000인 여러 clock_divider 모듈을 동시에 생성하고 각각의 출력을 LED에 연결한다.
- tb_clock_divider.sv

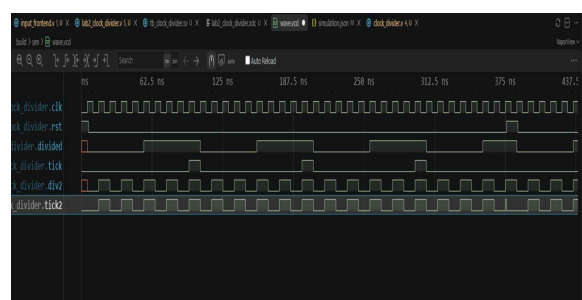
DIVISOR=10인 일반적인 분주기와 최소 분주값인 DIVISOR=2를 동시에 검사한다. 총 30개의 클록에 대해 duty cycle, 위상 및 tick을 매번 확인하고, 10분주 회로의 tick이 3회 사용되는지 검사한다.

pass 로그와 전체 파형

-pass 로그



-전체 파형



전체 시뮬레이션은 약 436ns까지 진행된다. 처음 0~5ns에서는 첫 번째 reset 클럭 에지가 아직 발생하지 않았기 때문에 일부 신호가 X인 미정 상태를 갖는다. 첫 번째 클럭 상승 에지 이후 reset에 의해 내부 상

LED[1]	100Hz	10ms	파형/장비
LED[2]	20Hz	50ms	주기 비교
LED[3]	1Hz	1s	육안으로
LED[4]	tick	폭1ms	파형/장비
LED[7:5]	0	OFF	육안으로

4. 실험 3 레지스터: 데이터 저장과 이동

제어 입력별 다음 상태 표와 동시 동적의 계산 과정

register_pair에는 두 개의 레지스터가 존재한다. stored는 입력을 저장하는 4비트 레지스터, value는 저장값을 전달받는 4비트 레지스터이다. 모두 clk 상승 에지에서 갱신된다. rst=1이면 두 레지스터를 먼저 0으로 만든다. load=1이면 data_in을 stored에 저장한다. transfer=1이면 에지 직전 stored를 value에 전달한다. 한 에지의 다음 상태는 다음의 표와 같다.

load와 transfer는 서로 독립적인 if 문으로 처리되기 때문에 두 신호가 모두 1인 경우에도 두 동작이 모두 실행된다. 또한 Verilog의 nonblocking assignment에서는 오른쪽 항의 값이 모두 clock edge 이전의 값을 기준으로 계산된다.

RTL, TB, simulation.json, XDC의 역할

- register_pair.v

두 개의 레지스터 stored와 value를 가지고 있고 클록의 상승 에지마다 rst, load, transfer의 상태에 따라 다음 값을 결정한다.

- input_frontend.v

FPGA 보드에서 입력되는 reset, button, DIP switch 신호를 내부 회로에서 안정적으로

사용할 수 있도록 처리할 수 있도록 한다. 외부 버튼과 스위치 신호를 주 clock에 동기화하며, 버튼은 일정 시간 동안 같은 값이 유지되는지를 확인하여 bounce에 의한 여러 번의 입력을 방지한다. 본 실험에서는 stable cycles 동안 입력이 안정된 경우에만 외부 입력을 받아들인다. 안정화 시간은 20 ms이다.

- lab2_register.v

실제 FPGA 보드와 내부 모듈을 연결하는 top module이다. input_frontend에서 생성된 press와 switches를 이용하여 register_pair의 제어 신호를 만든다. 따라서 단순히 DIP switch를 변경하는 것만으로 레지스터 값이 변하는 것이 아니라 원하는 switch 값을 먼저 설정한 뒤 버튼을 눌러 press가 발생해야 실제 동작이 수행된다.

- tb_register_pair.sv

register_pair가 설계한 대로 동작하는지를 자동으로 검사한다. 테스트 벤치에서는 clock을 생성한 뒤 총 7개의 상황을 순서대로 검사한다.

- simulation.json

VS Code에서 Icarus Verilog를 이용해 시뮬레이션할 때 위에서 설명한 3개의 RTL 파일을 source로 하고 tb_register_pair를 simulation top으로 사용한다.

- lab2_register.xdc

FPGA의 논리적인 Verilog port를 실제 보드의 물리적인 FPGA pin에 연결하고 I/O standard 및 clock 조건을 지정하는 제약 파일이다.

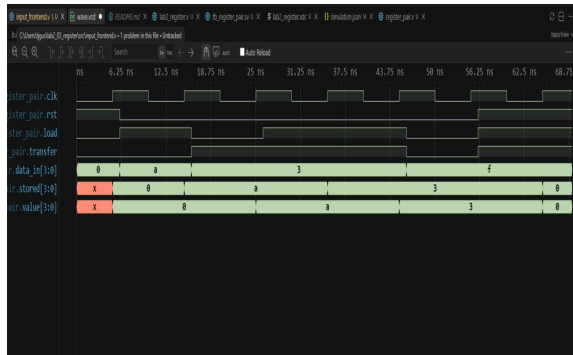
pass 로그와 전체 파형/확대 파형 해석

-pass 로그

```
Recording task in folder PROJECT: C:\Users\lgas\AppData\Local\Programs\Python\Python310\python.exe tools\lab2.py simulate
INFO: dumpfile wave.vcd opened for output.
INFO: PASSED register_pair checks?
C:\Users\lgas\lab2\lab2_register\lab2_register_pair.sv:31: $finish called at 6000 (ns)

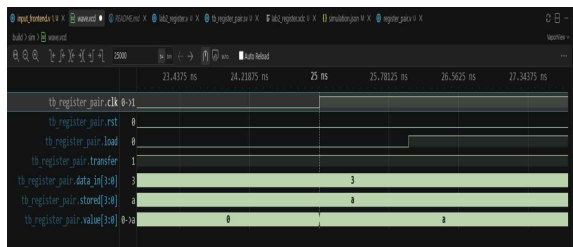
시뮬레이션 실행 완료. 전파표, 시가장서, 계층으로 계층의 결말 여부를 확인하세요.
INFO: C:\Users\lgas\lab2\lab2_register\lab2_register_pair.sv:31: $finish called at 6000 (ns)
WARNING: The simulation will be resumed by tasks, press any key to close it.
```

-전체 파형

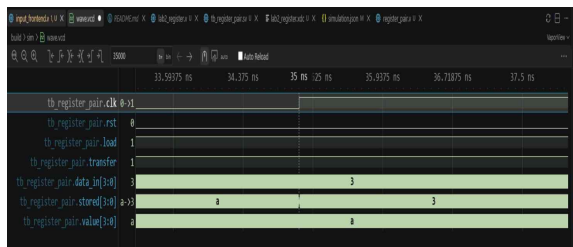


검사 시각	입력/동작	expected
6ns	reset	00
16ns	A저장	A0
26ns	data_in=3 stored 전달	AA
36ns	3 저장, 전달 수행	3A
46ns	다시 저장, 전달 수행	33
56ns	data_in=F 두 제어=0	33
66ns	reset	00

-25/35/45ns 확대 파형 해석

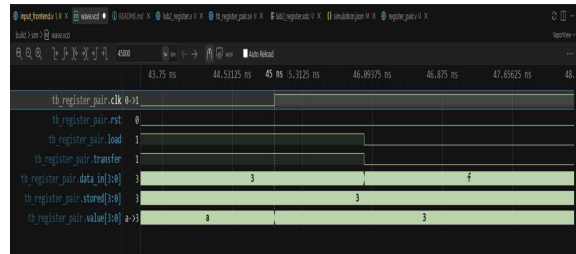


25ns를 보면 상승 에지가 있을 때 transfer가 1이기 때문에 stored에 있는 정보가 value로 이동한다. 따라서 0->a가 된다.



35ns를 보면 load와 transfer 모두 1이기

때문에 동시에 실행된다. 하지만 두 문장 모두 nonblocking assignment이기 때문에 이전의 값을 활용하여 stored는 3, value는 A의 값을 갖게 된다.



45ns일 때 상승 에지가 발생하므로 이제 stored에 있는 3이 value로 들어갈 수 있게 된다.

수정 실험: 저장값 대신 입력 전달

register_pair.v의 value <= stored를 value <= data_in으로 바꾼다. TB는 바꾸지 않는다. 이 실험을 시행하면 다음과 같다.

```

Running task in folder PROJECT: C:\Users\tjgus\AppData\Local\Programs\Python\Python38\python.exe tools\lab2.py simulator
RUN C:\msys64\mingw64\bin\perl.exe -g002 -h002 -f C:\Users\tjgus\lab2_03_register\src -s tb_register -o C:\Users\tjgus\lab2_03_register\build\sim\run-6a9fa2c1b6a945f8aff0f6589e46f7ef\sim.vvp C:\Users\tjgus\lab2_03_register\src\input_frontend.v C:\Users\tjgus\lab2_03_register\src\register_pair.v C:\Users\tjgus\lab2_03_register\src\register_pair.v C:\Users\tjgus\lab2_03_register\src\register_pair.v
VCD info: dumpfile wave.vcd opened for output.
LAB2_FAIL: transfer stored not live input C0000000
FAIL: C:\Users\tjgus\lab2_03_register\src\register_pair.v:31: check failed
Time: 26000. Scope: tb_register_pair_check
ERROR: 종료 코드 1: C:\Users\tjgus\lab2_03_register\build\sim\run-6a9fa2c1b6a945f8aff0f6589e46f7ef\sim.vvp simulation.log
The terminal process "C:\Users\tjgus\AppData\Local\Programs\Python\Python38\python.exe" "tools\lab2.py", "simulator" terminated with exit code: 1.
Terminal will be reused by tasks, press any key to close it.

```

위의 로그 사진을 보면 time=26000에 fail한 것을 볼 수 있는데 26ns 시점에서 정상적인 설계로 value=A가 되어야 한다. 하지만 변경한 코드에서는 data_in이 직접 value에 전달되므로 stored, value = A, 3이 되고 transfer stored not live input으로 검사에서 오류가 발생한다.

코드원복후 로그를 확인하면 다음과 같다.

```

ab2_03_register\src -I C:\Users\tjgus\lab2_03_register\sim -s tb_register -o C:\Users\tjgus\lab2_03_register\build\sim\run-6a9fa2c1b6a945f8aff0f6589e46f7ef\sim.vvp C:\Users\tjgus\lab2_03_register\src\input_frontend.v C:\Users\tjgus\lab2_03_register\src\register_pair.v C:\Users\tjgus\lab2_03_register\src\register_pair.v C:\Users\tjgus\lab2_03_register\src\register_pair.v
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS: register_pair checks=7
C:\Users\tjgus\lab2_03_register\sim\tb_register_pair.sv:31: $finish called at 66000 (1ps)
시뮬레이션 실행 완료. 진리표·자기검사·파형으로 회로의 정당 여부를 확인하세요.
WAVE C:\Users\tjgus\lab2_03_register\build\sim\wave.vcd
Terminal will be reused by tasks, press any key to close it.

```


보드의 입력 순서와 예상 LED 계산

보드 입력/출력	기능	예상 LED
K4	reset	00
N8	button (동작 실행)	A0
sw[0]	load	AA
sw[1]	transfer	3A
sw[7:4]	data_in[3:0]	33
LED[7:4]	stored	33
LED[3:0]	value	00

DIP SW1-SW8은 sw[7]-sw[0] 순서다.
sw[3:2]는 사용하지 않는다.

5. 실험 4 시프트 레지스터

enable, rst별 다음 상태 표와

8 → 4 → A → 5 계산

value[3:0]은 네 비트를 저장한다. enable=1인 clk 상승 에지마다 serial_in을 value[3]에 넣는다. 이전 value[3], value[2], value[1]은 각각 value[2], value[1], value[0]으로 이동한다. 이전 value[0]은 버린다. rst=1이면 0으로 초기화한다. rst=0, enable=0이면 입력이 변해도 저장값을 유지한다. 따라서 설명을 표로 정리하면 다음과 같다.

rst	enable	next value
1	don't care	0000
0	0	prev value
0	1	{serial_in, prev value[3:1]}

이런 상태 표를 기반으로 8 → 4 → A → 5를 계산하면 다음과 같다.

0000 -> 1000 (serial_in = 1)
1000 -> 0100 (serial_in = 0)
0100 -> 1010 (serial_in = 1)
1010 -> 0101 (serial_in = 0)

RTL, TB, simulation.json, XDC의 역할

- shift_register4.v

4비트 값을 저장하는 모듈이다. rst가 활성화되면 value를 0000으로 초기화한다. 이 RTL의 동작 원리는 위에 설명한 표와 같다.

- input_frontend.v

FPGA 보드에서 입력되는 reset, button, DIP switch 신호를 내부 회로에서 안정적으로 사용할 수 있도록 처리할 수 있도록 한다. 외부 버튼과 스위치 신호를 주 clock에 동기화하며, 버튼은 일정 시간 동안 같은 값이 유지되는지를 확인하여 bounce에 의한 여러 번의 입력을 방지한다. 본 실험에서는 stable cycles 동안 입력이 안정된 경우에만 외부 입력을 받아들인다. 안정화 시간은 20 ms이다.

- lab2_shift_register.v

실제 FPGA 보드와 내부 모듈을 연결하는 top module이다. input_frontend에서 생성된 reset, press, switches 신호를 shift_register4와 연결한다. 여기서 press는 enable이고 switches는 serial_in이다. 따라서 실제 보드에서는 SW1으로 입력할 0 또는 1을 결정하고 N8 버튼을 눌렀다 놓을 때마다 Shift Register가 한 단계 이동한다.

- tb_shift_register4.sv

shift_register4가 설계한 대로 동작하는지를 자동으로 검사한다. 테스트 벤치에서는 clock을 생성한 뒤 총 8개의 상황을 순서대로 검사한다.

- simulation.json

VS Code에서 Icarus Verilog를 이용해 시뮬레이션할 때 위에서 설명한 3개의 RTL 파일을 source로 하고 tb_shift_register4를 simulation top으로 사용한다.

- lab2_shift_register.xdc

FPGA의 논리적인 Verilog port를 실제 보드의 물리적인 FPGA pin에 연결하고 I/O standard 및 clock 조건을 지정하는 제약 파일이다.

	N8 한 번	
8	SW1=0, N8 한 번	00
9	SW1=0, N8 한 번	00

6. 실험 5 PISO

동작 표, 상태 전이, 타이밍을 예상

PISO는 Parallel In, Serial Out이다. load=1 인 상승 에지에서 data_in의 4bit를 value에 한 번에 저장한다. serial_out은 항상 value [3]이다. 먼저 현재 MSB를 읽고, enable=1 인 다음 상승 에지에서 {value[2:0], 0}으로 이동한다. 입력을 bit 3에 넣었던 앞 실험과 달리, 이번에는 저장값을 bit 3방향으로 옮기고 bit 0을 0으로 채운다.

다음의 표는 piso의 제어 우선순위다.

rst	load	enable	next value
1	don't care	don't care	0
0	1	don't care	data_in
0	0	1	{prev value[2:0], 0}
0	0	0	유효지

동작 예상을 해보자. 예를 들어서 1010을 읽는 시점을 살펴보자. 네 비트 1,0,1,0을 읽으려면 각 shift 예지 전에 출력값을 기록한다. shift 뒤에만 네 번 읽으면 첫 비트를 놓친다. 동작 예상 표는 다음과 같다.

관찰 시점	value	serial_out
load직후, shift전	1010	1
첫 shift 후	0100	0
둘째 shift 후	1000	1
셋째 shift 후	0000	0
넷째 shift 후	0000	0

RTL, TB, XDC 설정의 역할을 설명

- piso4.v

4-bit PISO의 실제 동작을 구현하는 핵심 RTL이다. 이 RTL의 동작 원리는 위에 설명한 표와 같다.

- input_frontend.v

FPGA 보드에서 입력되는 외부 신호를 내부 회로에서 안정적으로 사용할 수 있도록 처리할 수 있도록 한다. 외부 버튼과 스위치 신호를 주 clock에 동기화하며, 버튼은 일정 시간 동안 같은 값이 유지되는지를 확인하여 bounce에 의한 여러 번의 입력을 방지한다. 본 실험에서는 stable cycles 동안 입력이 안정된 경우에만 외부 입력을 받아들인다. 안정화 시간은 20ms이다.

- lab2_piso.v

실제 FPGA 보드와 내부 모듈을 연결하는 top module이다. input_frontend에서 생성된 reset, press, switches 신호를 piso4와 연결한다.

- tb_piso4.sv

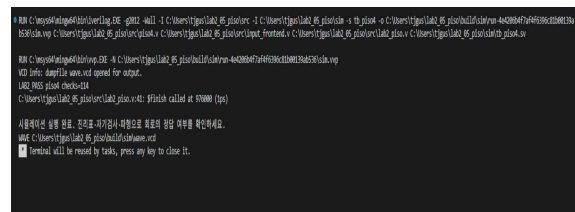
PISO4가 설계한 대로 동작하는지를 자동으로 검사한다. 테스트 벤치에서는 clock을 생성한 뒤 총 114개의 상황을 순서대로 검사한다.

- lab2 piso.xdc

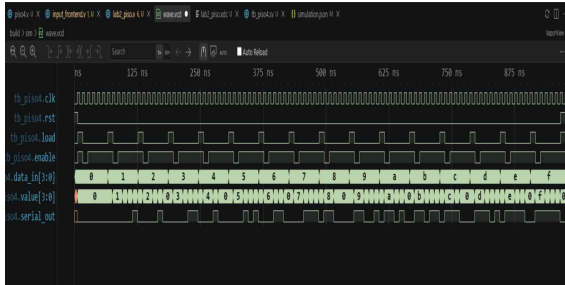
FPGA의 논리적인 Verilog port를 실제 보드의 물리적인 FPGA pin에 연결하고 I/O standard 및 clock 조건을 지정하는 제약 파일이다.

pass 로그와 전체 파형/ 구간 해석

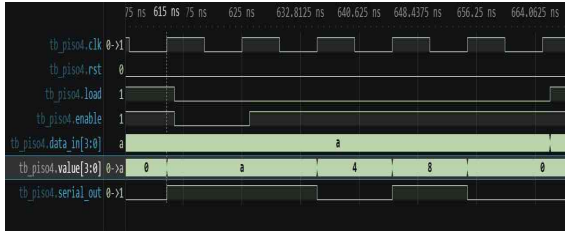
-pass 로그



-전체 파형



-입력·이전 상태·다음 상태를 설명



앞서 다룬 data_in = 1010(A) 구간을 살펴 보면 615ns에 load=1, data_in=1010이 입력으로 들어온다. 이전 상태 0000에서 다음 상태인 1010로 변함을 볼 수 있다. 이를 통해 입력이 병렬로 4비트가 한 번에 저장됨을 알 수 있다. 625ns에는 load와 enable이 모두 0으로 상태가 유지된다. 635~665ns에 enable이 1로 계속 shift가 되어 상태가 변함을 확인할 수 있다. 따라서 value가 10이 후에 4->8->0->0으로 변한다.

장비 클럭, 핀, 예상 LED 동작 정리

- 보드 장치

보드 장치	회로 신호	역할
B6	clk	1kHz 클럭
K4	rst	reset
SW8	0+button	shift
sw[0]	1+button	load
N8	button	shift/load 선택
LED[0]	LED[0]	serial_out
LED[3:1]	LED[3:1]	always OFF
LED[7:4]	LED[7:4]	value[3:0]

-예상 LED

value	동작	serial_	예상 LED
-------	----	---------	--------

		out	
1010	load	1	1010 0001
0100	shift 1회	0	0100 0000
1000	shift 2회	1	1000 0001
0000	shift 3회	0	0000 0000
0000	shift 4회	0	0000 0000

7. 실험 6 Moore 상태 머신

동작 표·상태 전이·타이밍을 예상

value는 2비트 상태 레지스터이면서 출력이다. S0=00, S1=01, S2=10으로 이름을 붙인다. 동작을 살펴보면 rst가 0일 때 enable과 advance가 모두 1인 상승 에지에서만 다음 상태로 간다. advance만 바꾸고 유효한 상승 에지를 주지 않으면 출력은 그대로다. 다음 상태가 입력에 의존하더라도 출력이 상태에만 의존하면 Moore다.

다음 표는 세 상태의 순환과 유지이다.

현재 상태	enable AND advance=1	그 외
S0 / 00	S1 / 01	S0
S1 / 01	S2 / 10	S1
S2 / 10	S0 / 00	S2

rst=1이면 다른 조건보다 먼저 00으로 초기화한다. case의 default는 10뿐 아니라 11도 처리하지만, 그 분기는 enable AND advance가 1일 때만 실행된다. case는 이전 상태를 읽고 <=로 다음 상태를 예약한다. 00 → 01 → 10 → 00의 각 화살표에 유효한 클럭 에지가 하나씩 필요하다. 보드의 N8 버튼은 clk가 아니라 enable 펄스를 만든다. SW1은 advance이며 두 제어를 구분한다.

RTL-TB-XDC·설정의 역할

- moore_cycle.v

moore cycle의 실제 동작을 구현하는 핵심 RTL이다. 상태 저장, 리셋, 상태 전이 및 출력 유지를 수행한다. 이 RTL의 동작 원리는 위에 설명한 표와 같다.

- input_frontend.v

FPGA 보드에서 입력되는 reset, button, DIP switch 신호를 내부 회로에서 안정적으로 사용할 수 있도록 처리할 수 있도록 한다. 외부 버튼과 스위치 신호를 주 clock에 동기화하며, 버튼은 일정 시간 동안 같은 값이 유지되는지를 확인하여 bounce에 의한 여러 번의 입력을 방지한다. 본 실험에서는 stable cycles 동안 입력이 안정된 경우에만 외부 입력을 받아들인다. 안정화 시간은 20 ms이다.

- lab2_moore.v

실제 FPGA 보드와 내부 모듈을 연결하는 top module이다. input_frontend에서 생성된 신호를 moore_cycle과 연결한다. 신호 연결은 다음과 같다. button은 상태 진형 버튼 입력, sw[7]은 DIP SW1에 해당하는 advance 입력, press는 버튼 처리 결과로 생성되는 코어의 enable이다.

- tb_moore_cycle.sv

moore_cycle이 설계한 대로 동작하는지를 자동으로 검사한다. 테스트 벤치에서는 clock을 생성한 뒤 총 23개의 상황을 순서대로 검사한다.

- simulation.json

VS Code에서 Icarus Verilog를 이용해 시뮬레이션할 때 위에서 설명한 3개의 RTL 파일을 source로 하고 tb_shift_register4를 simulation top으로 사용한다.

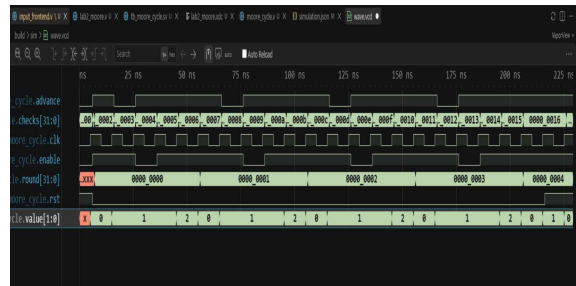
- lab2_moore.xdc

FPGA의 논리적인 Verilog port를 실제 보드의 물리적인 FPGA pin에 연결하고 I/O standard 및 clock 조건을 지정하는 제약 파일이다.

VS Code 파형을 확대해 입력, 이전 상태,

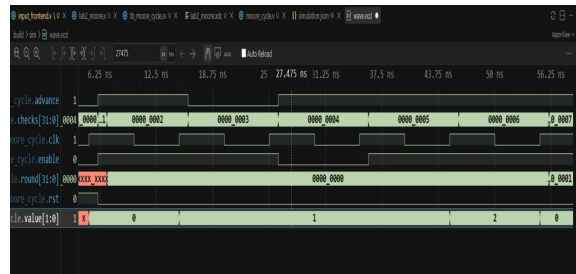
다음 상태를 설명

- 전체 파형



첫 순환의 다섯 에지 검사를 총 네 번 반복한다. 마지막 순환의 00 복귀 검사는 206 ns이다. 215ns 에지에서 다시 01로 간 뒤 rst=1을 설정한다. 226ns에 00 초기화를 검사한다.

- TB 첫 순환의 검사 시작



검사 시각	자극	value
6ns	reset	00
7ns	advance만 1로 변경	00 유지
16ns	유효 에지	01
26ns	advance=0	01 유지
36ns	enable=0	01 유지
46ns	두 제어 모두 1	10
56ns	다음 유효 에지	00

장비 클럭, 핀, 예상 LED 동작 정리

- 보드 장치 핀 조작

신호	FPGA 핀	기능
clk	B6	1 kHz 클럭

rst	K4	초기화 입력
button	N8	상태 진행 버튼
sw[7:0]	Y1, W3, U2, T1, W4, W1, V4, U4	DIP 스위치 입력
LED[7:0]	L4, P4, M2, N7, M7, M3, M1, N5	LED 출력

- 예상 LED 동작

순서	조작	예상 LED	해석
1	rst	00	S0
2	SW1=1 buttonX	00 유지	advance만 변경
3	button	01	S0->S1
4	SW1=0 button	01 유지	advance=0
5	SW1=1 button	10	S1->S2
6	button	00	S2->S0

8. 실험 7 Mealy 상태 머신

동작 표, 상태 전이, 타이밍 예상

Mealy 상태 머신은 현재 상태와 현재 입력을 함께 이용하여 출력을 결정하는 상태 머신이다. 이번 실험에서 state는 클록의 상승 에지에서만 변하는 1비트 레지스터이고, value는 state와 bit_in을 이용하여 계산되는 조합 출력이다. 따라서 state는 클록 에지가 있어야 변경되지만, value는 bit_in이 변경되면 클록 에지 없이도 바뀔 수 있다.

상태 전이와 출력에 따른 밀리 머신의 동작 예상표는 다음과 같다.

현재 state	bit_in	value	enable=1인 상승 에지 후 state
S0 / 0	0	00	S0

S0 / 0	1	10	S1
S1 / 1	0	00	S1
S1 / 1	1	01	S0

RTL, TB, XDC 설정의 역할

- mealy_toggle.v

mealy 상태 머신의 실제 동작을 구현하는 핵심 RTL이다. state 저장 및 value 생성을 수행한다. 이 RTL의 동작 원리는 위에 설명한 표와 같다.

- input_frontend.v

FPGA 보드에서 입력되는 reset, button, DIP switch 신호를 내부 회로에서 안정적으로 사용할 수 있도록 처리할 수 있도록 한다. 외부 버튼과 스위치 신호를 주 clock에 동기화하며, 버튼은 일정 시간 동안 같은 값이 유지되는지를 확인하여 bounce에 의한 여러 번의 입력을 방지한다. 본 실험에서는 stable cycles 동안 입력이 안정된 경우에만 외부 입력을 받아들인다. 안정화 시간은 20 ms이다.

- lab2_mealy.v

실제 FPGA 보드와 내부 모듈을 연결하는 top module이다. input_frontend에서 생성된 신호를 mealy 상태 머신과 연결하고 LED 출력을 생성한다.

- tb_mealy_toggle.sv

mealy 상태 머신이 설계한 대로 동작하는지를 자동으로 검사한다. 테스트 벤치에서는 clock을 생성한 뒤 총 11개의 상황을 순서대로 검사하고 67ns에 종료한다.

- simulation.json

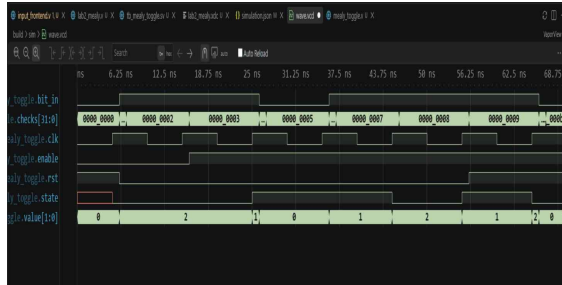
VS Code에서 Icarus Verilog를 이용해 시뮬레이션할 때 위에서 설명한 3개의 RTL 파일을 source로 하고 tb_mealy_toggle을 simulation top으로 사용한다.

- lab2_moore.xdc

FPGA의 논리적인 Verilog port를 실제 보드의 물리적인 FPGA pin에 연결하고 I/O standard 및 clock 조건을 지정하는 제약 파

일이다.

VS Code 파형을 확대해 입력, 이전 상태, 다음 상태를 설명



파형을 확대해서 6~36ns를 보면 시뮬레이션 결과는 다음과 같다.

검사 시각	자극/결과	{state, value}
6ns	reset, input=0	000
7ns	input=1 유지	010
16ns	enable=0 유지	010
26ns	enable=1 유지	101
27ns	input만 0	100
36ns	input=0 유지	100

6ns일 때 rst가 우선으로 000이 됨을 알 수 있다. S0일 때 input이 1이면 value는 10이 됨을 앞서 설명했다. enable이 0이면 상태 전이가 발생하지 않는다. 따라서 7ns에서 16ns까지 상태 변이는 발생하지 않았다. enable이 1이 되자 7ns부터 이어진 input 1이 들어와서 S0에서 S1으로 상태 전이가 발생한다. 다음으로 S1일 때 input이 0이면 value는 00이 되므로 100이 되게 된다. 36ns까지 input이 유지되므로 state, value값도 유지된다.

장비 클럭, 핀, 예상 LED 동작 정리

- 보드 핀의 역할

신호	FPGA 핀	역할
----	--------	----

clk	B6	1 kHz 클럭
rst	K4	reset
button	N8	상태 전이용 버튼
sw[7]	V1	bit_in, DIP SW1

- LED 동작 예상

state	SW1 (bit_in)	LED [2]	LED [1:0]	LED [2:0]
S0=0	0	0	00	000
S0=0	1	0	10	010
S1=1	0	1	00	100
S1=1	1	1	01	101

보드에서는 DIP SW1에 해당하는 sw[7]이 bit_in 입력으로 사용된다. SW1을 먼저 설정한 후 버튼을 눌러 상태를 전이시킨다. LED는 LED[2]가 현재 상태를 표시하고 LED [1:0]가 value를 표시한다.

9. 실험 8 7세그먼트 자리 스캔

동작 표, 상태 전이, 타이밍 예상

digits는 8개의 4비트 숫자를 묶은 32비트 입력이다. index는 3비트 값으로 0~7을 표현하며, index=0은 가장 낮은 4비트, index=7은 가장 높은 4비트를 선택한다. index는 3비트라 7 다음 증가에서 0으로 순환한다. nibble = digits » (index * 4)의 결과를 4비트에 대입하여 해당 숫자를 얻는다. select는 논리적인 active-high 자리 선택이다. 따라서 3:8 디코더처럼 행동한다. 다만 실제 COM 핀의 순서와 극성은 보드 모듈에서 변환하므로 seg_com으로 출력한다.

다음의 표는 상태 전이를 나타낸 것이다.

prev state	enable	next index	next blank	동작
reset	X	0	1	all OFF

blank =1	1	유지	0	현재 자리 활성
blank =0	1	index =1	1	다음 자리 준비 및 소등
X	0	유지	유지	현재 상태 유지

실제 보드에서는 enable=1로 고정되어 있고 메인 클록은 1 kHz를 사용한다. 따라서 한 클록의 시간은 1ms이다. 한 자리마다 활성 1ms와 blank 1ms로 2ms가 필요하다. 8 자리를 모두 돌기 위해서 16ms가 필요하므로 각 자리가 다시 켜지는 주파수는 62.5Hz가 된다.

RTL, TB, XDC 설정의 역할

- segment_scan8.v

실제 자리 스캔을 수행하는 핵심 모듈이다. index와 blank 상태를 관리하며, digits에서 현재 자리의 4비트 숫자를 가져와 segments로 변환한다. 또한 blank 상태에 따라 select를 출력한다.

- input_frontend.v

보드에서 들어오는 rst, button, sw 입력을 FPGA 내부에서 사용하기 위한 입력 처리 모듈이다. reset은 비동기적으로 입력되지만 해제는 메인 클록에 맞추어 이루어지며, button과 switch 역시 동기화 과정을 거친다. 다만 이번 7-Segment 자동 스캔에서는 board top에서 segment_scan8의 enable을 항상 1'b1로 연결하기 때문에 N8 버튼을 눌러서 자리를 이동시키는 방식이 아니다.

- lab2_segment_scan.v

실제 FPGA 보드에서 사용하는 Top module이다. 입력 스위치와 고정 숫자를 결합하여 다음과 같이 digits를 구성한다. digits = {7654321, SW[7:4]}. 따라서 index=0에 해당

하는 첫 번째 표시 숫자만 sw[7:4]에 의해 변경되고, 이후 자리는 차례대로 1, 2, 3, 4, 5, 6, 7이 표시된다. 또한 내부 select는 active-high이지만 실제 COM 출력은 극성과 순서가 다르므로 seg_com으로 변환한다.

- tb_segment_scan8.sv

mealy 상태 머신이 설계한 대로 동작하는지를 자동으로 검사한다. 테스트 벤치에서는 clock을 생성한 뒤 총 194개의 상황을 순서대로 검사하고 1306ns에 종료한다.

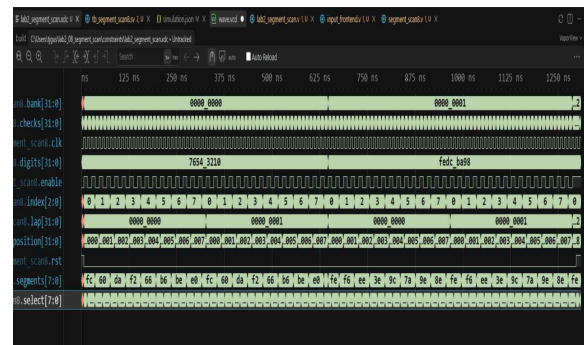
- simulation.json

VS Code에서 Icarus Verilog를 이용해 시뮬레이션할 때 위에서 설명한 3개의 RTL 파일을 source로 하고 tb_segment_scan8을 simulation top으로 사용한다.

- lab2_moore.xdc

FPGA의 논리적인 Verilog port를 실제 보드의 물리적인 FPGA pin에 연결하고 I/O standard 및 clock 조건을 지정하는 제약 파일이다.

VS Code 파형을 확대해 입력, 이전 상태, 다음 상태를 설명



검사 시각	enable	index	select	동작
6ns	0	0	00	blank
16ns	1	0	01	0활성
26ns	0	0	01	유지
36ns	1	1	00	blank
46ns	0	1	00	유지
56ns	1	1	02	1활성

6ns에 rst=1을 입력해서 모든 자리가 꺼진다. 16ns에 enable=1로 select=01이 되고 digits[3:0]=0이므로 segments=FC(0)이 출력

된다. enable이 0이므로 값이 유지되고 36ns에 enable이 1이 되지만 blank로 값이 유지되면서 숫자 0에서 1로 이동하는 구간이다. 56ns에 다시 enable이 1이 되면 select=02가 되고 segments=60으로 1이 출력된다.

장비 클록, 핀, 예상 LED 동작 정리

보드에서는 메인 클록을 1 kHz로 사용하며 스캔의 enable은 항상 1로 설정된다. 따라서 버튼을 누르지 않아도 8개의 자리가 자동으로 계속 순환한다. 앞서 설명한 것과 같이 클록의 주기는 1ms고 반복 주파수는 62.5Hz가 된다.

- 주요 입력 핀

신호	pin	역할
clk	B6	1 kHz 클록
rst	K4	reset
button	N8	버튼 입력
sw[7:0]	U4, V4, W1, W4, T1, U2, W3, Y1	DIP switch

- LED 동작 예상

LED[7:3]은 항상 0이고, LED[2:0]이 현재 스캔 중인 index를 3비트 이진수로 표시한다.

index	led[7:0]	켜지는 LED
0	0000_0000	없음
1	0000_0001	LED0
2	0000_0010	LED1
3	0000_0011	LED1, LED0
4	0000_0100	LED2
5	0000_0101	LED2, LED0
6	0000_0110	LED2, LED1
7	0000_0111	LED2, LED1, LED0

10. 참고문헌

- 전전설 교안 LAB2